ZitPit: The Artifact Firewall for Agentic Software Supply Chains

Turning First-Seen Code into Policy Events

Jepson Taylor
jepson@veox.ai
Chris Brousseau
chris@veox.ai
Jordan Hildebrandt
j@veox.ai
Kelli Quinn
kelli@veox.ai
VEOX Research Group
Open-source research draft
https://github.com/jeppsontaylor/zitpit

Abstract

AI IDEs and coding agents compress search, dependency intake, workspace open, and execution into one low-observability loop. In that setting, a repository clone, package install, workflow reference, shell bootstrap, or repo-scoped agent configuration can cross from discovery to execution before a human reaches a durable review checkpoint. ZitPit is a mandatory mediation layer for consumer-side software intake in agentic development environments. It resolves external requests to immutable identities when possible, creates durable policy events before protected-host execution, routes first-seen artifacts into a controlled evidence lane, and accelerates approved artifacts through local cache and in-memory hot cache. This paper defines the system's claim boundary, threat model, policy model, and trust inputs, and reports preliminary public evidence from repeated Git smart-HTTP intake benchmarks plus a broader attack-family coverage matrix. The paper's central claim is narrow and enforceable: first-seen external code becomes a policy event before it becomes execution on a protected host.

Index Terms—software supply chain security, AI agents, artifact firewall, provenance, quarantine, governed execution, observability

I. Introduction

Imagine a developer asking an AI IDE to "set up this repo and wire in observability." In the next few seconds the tool may search for a package, clone a repository, open project memory files, read repo-scoped agent settings, start a devcontainer, install dependencies, and invoke helper scripts. The human often sees prompts, shell output, or a final working state, but not a durable checkpoint where newly discovered external code has been resolved, reviewed, and granted execution rights under a stable policy. In practice the operator is often approving *intent* rather than concrete external artifacts.

What internet culture loosely calls "vibe coding" is more precisely *agent-mediated development under weak operator visibility*. The security problem is not only that a package might be malicious. It is that discovery, fetch, configuration, installation, repo-open state, and execution now collapse into one conversational loop with poor artifact-level observability. Claude Code documentation reflects this operational reality directly through npm-based installation, project-scoped MCP configuration, hooks, devcontainer guidance, and working-tree memory files such as `CLAUDE.md` [1]–[5]. GitHub similarly documents that third-party workflow actions should be pinned to immutable SHAs because mutable references create trust ambiguity exactly where automation is strongest [6].

This pressure is especially acute for small teams. Large enterprises may have internal mirrors, provenance pipelines, and release engineering staff. Smaller organizations and solo developers often have the same speed incentives without those controls. OpenSSF's guidance for AI code assistants explicitly calls out prompt injection, package confusion, and automation risks that grow when tools act faster than humans can verify [7]. The result is a three-part collapse: time from discovery to execution shrinks, artifact-level visibility degrades, and agents inherit cross-domain authority to search, fetch, open, install, and run.

Recent incidents show why this matters. The Axios npm compromise of March 31, 2026 demonstrated how a short-lived install-time compromise can become an immediate downstream problem for developer and CI environments that inherit trust too early [8], [9]. More broadly, the modern intake surface includes repo-open configuration and execution points that older supply-chain discussions often treated as inert: npm lifecycle scripts, Rust `build.rs`, repo-local hooks, MCP server definitions, devcontainers, startup tasks, and mutable workflow references [3], [10], [11].

Why traditional review collapses in agentic development. Older software intake assumed a sequence such as *discover* → *inspect* → *decide* → *install* → *execute*. Agentic development often inverts that into *discover* → *install/open/run* → *inspect later*. ZitPit exists to restore a durable decision and audit boundary at the artifact layer rather than at the final command or prompt layer.

ZitPit is built around a narrow claim with operational consequences: first-seen external code should become a policy event before it becomes an execution event, and approved immutable artifacts should stay on a faster path than unmanaged public fetch. This paper makes three contributions: it defines a crisp claim boundary for consumer-side agentic software intake; it describes a mediated architecture and policy model that restore artifact-level observability; and it reports preliminary but public evidence that the governed path can remain fast while broader attack families are made explicit and auditable.

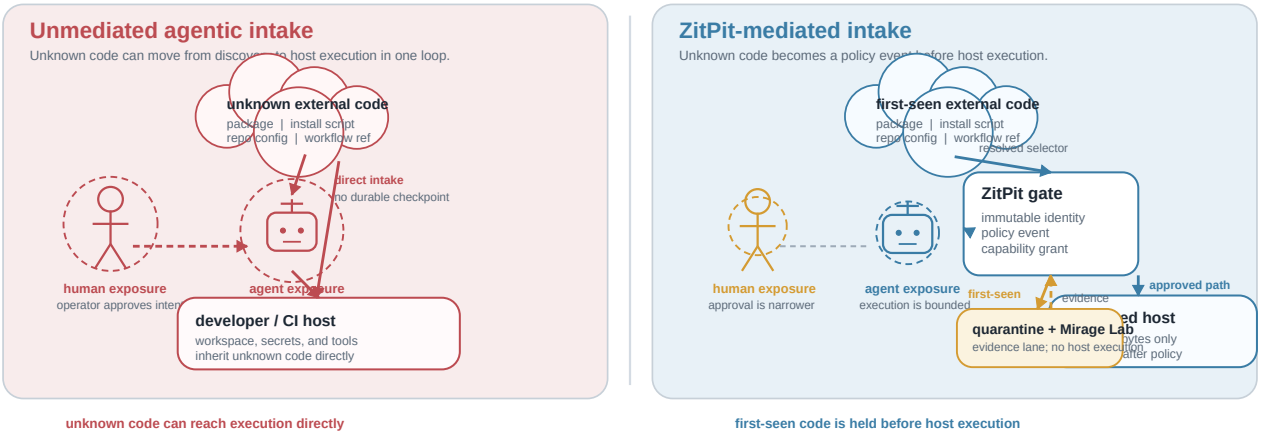

Fig. 1. Without mediation, unknown code can move from discovery to execution in one loop; with ZitPit, first-seen external code becomes a policy event before protected-host execution.

II. Scope and Claim Boundary

ZitPit intentionally makes a narrower claim than "solve software supply-chain security." The paper is about consumer-side mediation: the point at which external code, workflow references, or repo-open state cross into execution rights on a protected developer or CI host. Producer-side release hygiene still matters, but it is treated as complementary rather than as the main evaluated control.

Consumer intake (in scope)	Repository fetches, package installs, mutable workflow refs, raw bootstraps, and repo-open execution surfaces are mediated before protected-host execution.
Producer release (complementary)	Trusted publishing, attestations, and release firewalls matter, but they are not the central evaluated control in this paper.

In-scope claim. First-seen external code does not execute on protected hosts before policy evaluation and capability grant.
Non-claims. Trusted maintainers may still be malicious; producer-side release leaks need publish controls; unsupported off-path fetches are outside the guarantee; sandbox silence does not imply safety.

III. Threat Model and Definitions

Definitions. An *artifact* is the external code object or repo-scoped execution bundle being mediated: a Git ref target, package tarball, wheel, crate source, workflow action, raw installer payload, or repo-open configuration surface. *First-seen* means first observed by the ZitPit-mediated trust domain for a given immutable identity, not merely first seen by one process or laptop. A *policy event* is the durable record created when a selector is resolved, evaluated, and granted or denied capability. A *protected host* is a developer or CI environment whose execution rights are gated by ZitPit. A *repo-open surface* is working-tree content that can trigger execution or tool behavior simply by opening or operating in the repository.

TABLE I
Threat Model and Assumptions

Asset	Adversary	In scope	Control point	Residual risk
Host execution rights	Malicious upstream package or repo	Yes	Immutable resolution + policy event	Trusted publisher compromise remains possible
CI workflow integrity	Mutable action refs or unsafe workflow reuse	Yes	Full-SHA policy and mediated fetch	Off-runner reuse outside mediation is not covered
Local secrets and state	Repo-open hooks, MCP, devcontainers, startup tasks	Part.	Capability gate before repo-open execution	IDE integration depth varies by tool
Network egress	Raw bootstrap downloads or installer scripts	Part.	Controlled cold lane and mediated fetch path	Direct unmanaged egress is out of scope unless separately blocked
Release hygiene	Wrong registry publish or source-map leak	Ext.	Trusted publishing and release controls	Not the paper's primary evaluated guarantee

The threat model assumes that protected hosts are not already fully compromised, that mediated paths are mandatory or transparently redirected, and that the approval store and local trust root remain intact. The system's primary goals are to resolve external code to immutable identities before execution, grant execution through capability-scoped policy rather than ambient trust, and record artifact-level audit events that survive beyond a single prompt or shell command. The non-goals are equally important: ZitPit does not claim to eliminate software supply-chain risk universally, to prove trusted publishers benign, or to convert sandbox quiet into a proof of safety.

IV. Architecture and Policy

A. Acquire (Implemented)

The current implementation is strongest on intake mediation. Requests from agents, IDEs, or CI runners first hit a gateway that resolves selectors onto the strongest available immutable identity. For Git this means the smart-HTTP path and resolved

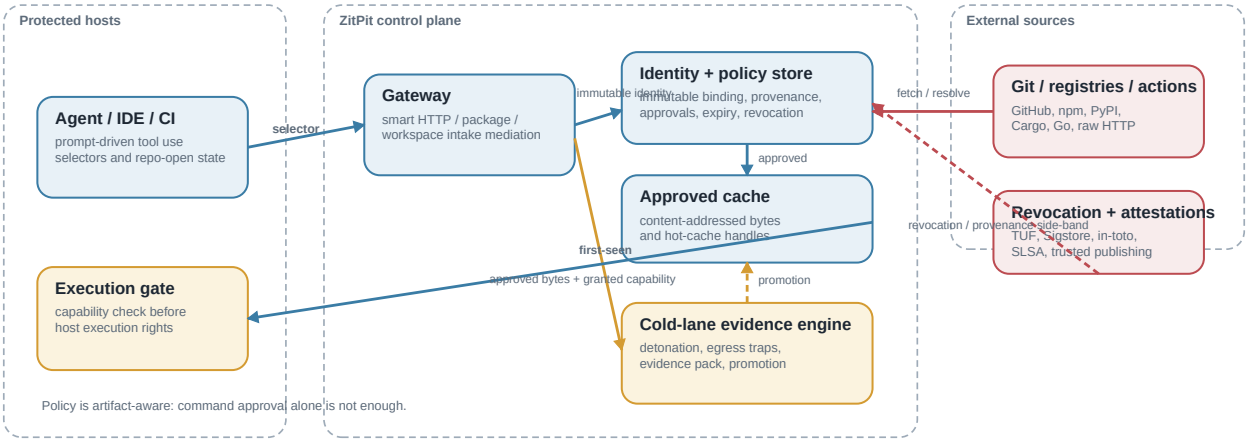

Fig. 2. Trust boundaries and control flow. The architecture is centered on mediated intake, not on release branding. Policy, provenance, cache, cold-lane evidence, and revocation live in the control plane; execution rights are granted only after immutable binding and verdict evaluation.

commit identity; for other surfaces it means exact digests, pinned SHAs, or equivalent immutable selectors wherever the ecosystem allows it. Floating tags, branch heads, and `latest` are treated as policy exceptions rather than default trust.

B. Controlled Build Lane (Partial)

Many ecosystems blur fetch and execution into one step. npm lifecycle scripts, Python source distributions, Cargo build scripts, and shell installers can all execute during installation [10], [11]. ZitPit therefore separates acquisition from build. First-seen or policy-sensitive artifacts fall into Mirage Lab, the system's *cold-lane evidence engine*, before those execution surfaces touch a protected host.

C. Agent Observability and Decision Boundaries (Partial)

Current AI IDE controls are often command-aware, not artifact-aware. They may ask whether a shell command or tool call may run, but not whether newly discovered external code has earned execution rights under immutable identity, provenance, freshness, and capability policy [3], [12]. ZitPit treats repo-open state as supply-chain input for this reason: MCP definitions, hooks, devcontainers, memory files, startup tasks, and similar working-tree artifacts can cause execution before a human reads the repository.

This matters for deployment realism. The operator should be able to stay inside familiar environments such as Cursor or other IDE terminals while still getting a clear signal that intake is being mediated and that the protected path is active. The point is not product integration as advertising; it is that a mandatory intake boundary is easier to adopt when it appears as a visible safety state inside tools developers already use.

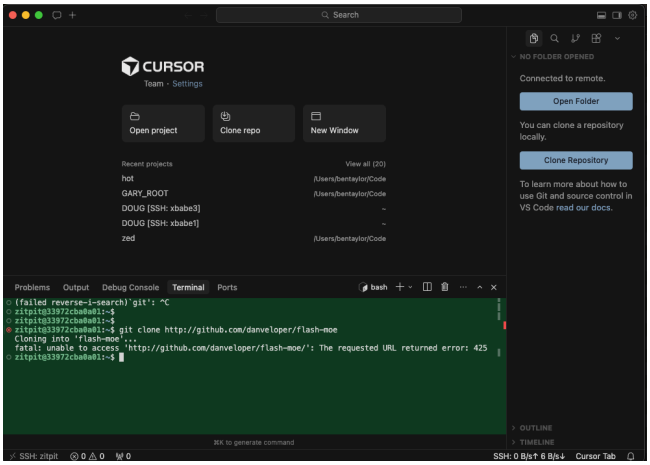

Fig. 3. Example deployment inside an existing IDE terminal. The mediation signal is conveyed through terminal-state color and status output rather than IDE-specific branding, so protected intake remains visible without requiring a custom editor experience or implying vendor endorsement.

Artifact policy event fields. initiator or agent session; requested selector; resolved immutable identity; provenance result; verdict; evidence ID; granted capability; timestamp; host or CI context. This is the missing audit boundary in many agentic workflows: a durable, artifact-aware record of why execution rights were granted.

D. Publish-Side Controls (Extension)

Publish-side controls matter, but they are treated here as a side-band extension rather than a central stage of the evaluated system. Release firewalls, trusted publishing, and artifact attestations help with source-map leaks, wrong-registry

TABLE II
CAPABILITY-SCOPED VERDICTS

Verdict	Typical use	Execution conditions
FETCH_ONLY	First-seen dependency or repo bundle	Download allowed; protected-host execution denied
BUILD_NO_NETWORK	sdist build, build.rs, installer analysis	Controlled lane only; no general egress
TEST_NO_SECRETS	Benign package or repo validation	Isolated test lane; no sensitive material
RUN_DEV	Approved developer dependency	Protected-host execution allowed under freshness policy
RUN_CI	Approved CI dependency or action	CI execution allowed under stronger identity and expiry checks
BLOCKED	Malicious, unsupported, or stale input	Denied pending expiry, recall, or operator action

publishes, and workflow drift, but they solve a different part of the problem than the intake contract defended in this paper.

Design invariants. (1) First-seen code is a policy event. (2) Immutable identity comes before protected-host execution. (3) The safe path must remain fast enough that developers and agents have less incentive to bypass the control plane.

E. Capability-Scoped Policy

Binary allow-or-block logic is too coarse for agentic development. ZitPit uses capability-scoped verdicts so that fetching bytes, building in quarantine, testing without secrets, and executing on a protected host are separated decisions rather than one inherited permission.

Lifecycle. Unknown → FETCH_ONLY → BUILD_NO_NETWORK or TEST_NO_SECRETS → RUN_DEV or RUN_CI → Expired, Revoked, or BLOCKED. Promotion depends on immutable binding, provenance or freshness signals, and cold-lane evidence when required.

Digest equality matters because it creates immutable identity, but identity alone is not provenance. ZitPit is designed to consume stronger trust signals when they exist: TUF-style freshness and anti-rollback semantics [13], [14], Sigstore transparency-backed signing [15], in-toto attestations [16], [17], SLSA provenance [18], GitHub artifact attestations [19], npm trusted publishing [20], and PyPI attestations and trusted publishing paths [21], [22]. The system's role is enforcement and mediation: it consumes identity and provenance signals where available and compensates with explicit policy when they are absent.

V. Preliminary Evaluation

The current evaluation is intentionally split into two proof obligations. The first is *deployability*: can the governed path remain fast enough for real developer and CI use? The second is *security coverage*: which attack families are actually mediated today, and how honest is the remaining boundary?

A. Performance and Deployability

The public benchmark harness measures the Git smart-HTTP intake path rather than a full repository clone. Specifically, it times repeated `git ls-remote / info/refs` style mediation across five public repositories: `git`, `go`, `node`, `cpython`, and `terraform`. Each repository is measured in three modes: direct upstream request (`web`), approved disk-cache hit (`cache`), and approved in-memory hot-cache hit (`hot-cache`) [23]. The current public snapshot uses $N = 5$ samples per repository and reports medians plus p95 values.

The current result is modest but operationally important. Repository-level web medians range from 433 to 1062 ms, approved disk-cache medians from 32 to 44 ms, and hot-cache medians from 13 to 16 ms. Even in this small public run, the safe path is not the slow path. That matters because a mandatory intake control will be bypassed under real development pressure if approved artifacts are slower than direct public fetch.

B. Security Coverage and Attack Families

Three representative attack families show the intended boundary. First, install-time package attacks should become policy events before lifecycle hooks execute on the protected host. Second, build-time execution such as Rust `build.rs` should be granted only in a controlled lane rather than inherited from fetch. Third, repo-open execution surfaces should be treated as supply-chain input, not inert project metadata. The coverage matrix is intentionally public because the credibility of the paper depends on visible support boundaries rather than implied universality.

This is still preliminary evaluation. The current paper does not claim a full cross-ecosystem empirical study; it claims that the deployability argument is already visible in repeated public latency data and that the security claim boundary is explicit enough to audit, extend, and attack in public.

VI. Limitations and Next Proof Obligations

Trusted publisher compromise. ZitPit does not make trusted maintainers harmless. A malicious publisher with valid identity can still ship bad code; provenance, freshness, recall, and external intelligence remain necessary.

Unsupported ingress and bypass. Any unmanaged path that fetches code outside the mediated domain weakens the guarantee. The correct status for those paths is "unsupported" rather than "secure enough." This is especially important for browser downloads, vendored code drops, and local copies that never cross the intake boundary.

Evidence limits. The cold-lane evidence engine improves ordering and observability, but it is not a safety oracle. Dynamic analysis can miss dormant or trigger-dependent behavior, and a quiet run cannot prove future benign behavior.

Operational burden and availability. Policy systems create approval workload and potential bottlenecks. A mediation layer also becomes part of the trusted computing base and part of the availability story.

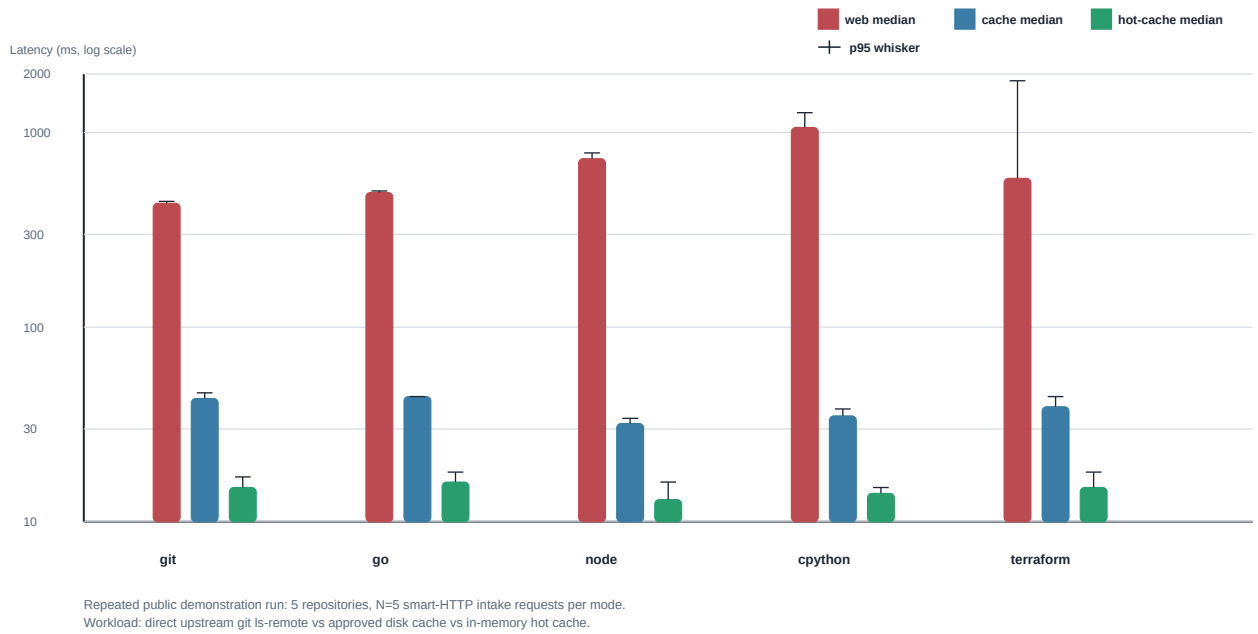

Fig. 4. Preliminary deployability result for the Git smart-HTTP intake path. Bars show repository-level medians across $N = 5$ samples; whiskers show p95. Repository-level web medians ranged from 433–1062 ms, cache medians from 32–44 ms, and hot-cache medians from 13–16 ms. The workload is mediated smart-HTTP intake, not a full clone.

TABLE III
ATTACK-FAMILY COVERAGE MATRIX

Surface	First execution boundary	Control point	Status	Evidence source	Residual risk
Git clone / fetch	checkout, follow-on build, or workspace open	smart-HTTP mediation and immutable ref binding	Implemented	repeated public benchmark harness [23]	unmanaged Git paths outside mediation
npm lifecycle scripts	install or postinstall hook	cold-lane build / execution separation	Partial	benchmark matrix + battle packs [24]	native package-manager integration still incomplete
Python sdist / startup	build hook or startup import path	controlled build lane and policy gate	Partial	benchmark matrix + scenario packs [24]	wheel/sdist mediation not universal yet
Cargo `build.rs`	compile-time build script	controlled build lane with no-network verdicts	Partial	benchmark matrix + Cargo scenarios [24]	broader Cargo-native interception remains future work
GitHub Actions mutable refs	workflow runner	full-SHA policy and selector resolution	Partial	workflow hardening scenarios [24]	off-runner use or unmediated workflows
Raw `curl | bash`	shell execution	mediated fetch path and cold-lane detonation	Partial	shell installer scenarios [24]	direct unmanaged egress if host bypasses intake control
Repo-open surfaces (`.mcp.json`, hooks, devcontainers)	workspace open or tool startup	workspace policy + execution gate	Partial	workspace / repo-open scenarios [24]	enforcement depth varies by IDE and agent runtime

Current proof obligations. The next technical step is not bigger rhetoric. It is broader mediation coverage, stronger provenance consumption, repeated multi-run evaluation across more ecosystems, clearer CI fail-closed evidence, and tighter publish-side controls where release hygiene matters.

VII. Related Work

ZitPit overlaps with several established lines of work, but it is not reducible to any one of them. Package mirrors and internal artifact repositories improve locality and availability, yet they do not by themselves create artifact-level policy events or capability-scoped host execution. Provenance systems such as TUF, in-toto, SLSA, Sigstore, GitHub artifact attestations, npm trusted publishing, and PyPI attestations provide the vocabulary and evidence needed to reason about publication identity and freshness [13]–[22]. ZitPit is intended to consume and enforce those signals when available, not to replace them with a parallel trust universe.

The system also differs from sandboxing and honeypot work. Mirage Lab is useful because it creates evidence and ordering, not because detonation proves safety. Likewise, ZitPit differs from command-centric AI IDE approval systems. Command approvals operate at the level of terminal or tool invocations; ZitPit aims to restore a durable artifact-level trust decision in workflows where repository open, transitive installation, and agent-controlled tool use can outrun human review [3], [12].

VIII. Conclusion

ZitPit starts from a practical observation about agentic development: the interval between discovering external code and executing external code is collapsing toward zero. That collapse changes the default security problem. What used to be a repository review problem is now an admission-control problem spanning registries, raw downloads, workflow engines, and repo-open execution surfaces.

The paper's contract is intentionally narrow. First-seen external code becomes a policy event before it becomes execution on a protected host. Approved immutable artifacts stay on a faster path through cache and hot cache. Everything else in the paper supports that claim: the scope boundary, the threat model, the control-plane architecture, the capability model, the preliminary evaluation, and the explicit limitations. The next step is not bigger claims. It is broader mediation coverage, stronger evidence, and sharper proof that unsafe first-seen software stays off protected hosts until it has earned capability through policy.

References

[1] Anthropic, "Claude code: Getting started," https://docs.anthropic.com/en/docs/claude-code/getting-started, 2026, accessed 2026-04-02.

[2] ——, "Claude code: Model context protocol (MCP)," https://docs.anthropic.com/en/docs/claude-code/mcp, 2026, accessed 2026-04-02.

[3] ——, "Claude code: Hooks reference," https://docs.anthropic.com/en/docs/claude-code/hooks, 2026, accessed 2026-04-02.

[4] ——, "Claude code: Manage claude's memory," https://docs.anthropic.com/en/docs/claude-code/memory, 2026, accessed 2026-04-02.

[5] ——, "Claude code: Devcontainer reference," https://docs.anthropic.com/en/docs/claude-code/devcontainer, 2026, accessed 2026-04-02.

[6] GitHub Docs, "Security hardening for GitHub Actions," https://docs.github.com/en/actions/security-guides/security-hardening-for-github-actions, 2026, accessed 2026-04-02.

[7] OpenSSF, "Security-focused guide for AI code assistants," https://openssf.org/wp-content/uploads/2025/07/Security-Focused-Guide-for-AI-Code-Assistants.pdf, 2025, accessed 2026-04-02.

[8] Datadog Security Labs, "Axios package compromise: Install-time malware in a widely used npm dependency," https://securitylabs.datadoghq.com/articles/axios-npm-compromise/, 2026, accessed 2026-04-02.

[9] Snyk, "Axios npm compromise: What happened and what to do," https://snyk.io/blog/axios-npm-compromise/, 2026, accessed 2026-04-02.

[10] npm, Inc., "package.json," https://docs.npmjs.com/cli/v11/configuring-npm/package-json, 2026, accessed 2026-04-02.

[11] The Rust Project, "The cargo book: Build scripts," https://doc.rust-lang.org/cargo/reference/build-scripts.html, 2026, accessed 2026-04-02.

[12] Anthropic, "Claude code: Security," https://docs.anthropic.com/en/docs/claude-code/security, 2026, accessed 2026-04-02.

[13] The Update Framework, "The update framework documentation," https://theupdateframework.io/, 2026, accessed 2026-04-02.

[14] J. Cappos, J. Samuel, S. Baker *et al.*, "Diplomat: Using delegations to protect community repositories," in *13th USENIX Symposium on Networked Systems Design and Implementation (NSDI 16)*, 2016, pp. 567–581. [Online]. Available: https://www.usenix.org/conference/nsdi16/technical-sessions/presentation/kaptchuk

[15] Sigstore Project, "Sigstore documentation," https://docs.sigstore.dev/, 2026, accessed 2026-04-02.

[16] in-toto Project, "in-toto," https://in-toto.io/, 2026, accessed 2026-04-02.

[17] S. Torres-Arias, A. Ammula, R. Curtmola, and J. Cappos, "in-toto: Providing farm-to-table guarantees for bits and bytes," in *28th USENIX Security Symposium (USENIX Security 19)*, 2019, pp. 1393–1410. [Online]. Available: https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias

[18] SLSA, "Supply-chain levels for software artifacts," https://slsa.dev/, 2026, accessed 2026-04-02.

[19] GitHub Docs, "Use artifact attestations," https://docs.github.com/en/actions/security-guides/use-artifact-attestations, 2026, accessed 2026-04-02.

[20] npm, Inc., "Trusted publishing with OIDC," https://docs.npmjs.com/trusted-publishers, 2026, accessed 2026-04-02.

[21] PyPI Docs, "Pypi publish attestation (v1)," https://docs.pypi.org/attestations/publish/v1/, 2026, accessed 2026-04-02.

[22] ——, "Publishing to pypi with a trusted publisher," https://docs.pypi.org/trusted-publishers/, 2026, accessed 2026-04-02.

[23] ZitPit, "Zitpit benchmark snapshot: Five public git repositories," Repository artifact, 2026, source file: `docs/benchmarks/latest.json`, accessed 2026-04-02.

[24] ——, "Zitpit benchmark matrix," Repository artifact, 2026, source file: `BENCHMARKS.md`, accessed 2026-04-02.